



Deployment: Production-Ready Data Science

Introdução

Prof. Donald Neumann, PhD.
Março de 2026

Contatos:
donaldn@insper.edu.br

Além de professor, sou consultor e lidero uma equipe de dados & AI

Quem sou?

- Chief Data & AI Officer @ Arvo
- PhD. Administração
Supply Chain Management
Universität Stuttgart
- M.Sc. Eng. de Sistemas
Inteligência Artificial Aplicada
PUCPR
- Administração de Empresas
UFPR
- Eng. Computação
PUCPR





Por que essa disciplina?

Motivação

- Como você/seu time constrói suas soluções de ciência de dados?
- O que significa deploy para você/seu time?
- Quais são as dificuldades que você/seu time encontram?

Por que essa disciplina?

Motivação

RQ7. *How reproducible are notebooks?*

Answer: We were able to successfully run 24.11% of the unambiguous execution order Python notebooks. This number is close to the results of a previous reproducibility study [32] about general computer systems research (24.9%). However, the rate is way smaller (4.03%) when we count only notebooks that produce the same results. The most common causes of failures were related to missing dependencies, the presence of hidden states and out-of-order executions, and data accessibility.

A Large-scale Study about Quality and Reproducibility of Jupyter Notebooks

João Felipe Pimentel*, Leonardo Murta*, Vanessa Braganholo*, and Juliana Freire†

*Universidade Federal Fluminense
Niterói, Brazil
{jpimentel, leomurta, vanessa}@ic.uff.br
†New York University
New York, USA
juliana.freire@nyu.edu

Abstract—Jupyter Notebooks have been widely adopted by many different communities, both in science and industry. They support the creation of literate programming documents that combine code, text, and execution results with visualizations and all sorts of rich media. The self-documenting aspects and the ability to reproduce results have been touted as significant benefits of notebooks. At the same time, there has been growing criticism that the way notebooks are being used leads to unexpected behavior, encourage poor coding practices, and that their results can be hard to reproduce. To understand good and bad practices used in the development of real notebooks, we studied 1.4 million notebooks from GitHub. We present a detailed analysis of their characteristics that impact reproducibility. We also propose a set of best practices that can improve the rate of reproducibility and discuss open challenges that require further research and development.

Index Terms—jupyter notebook, github, reproducibility

its library dependencies with associated versions, which can make it hard (or even impossible) to reproduce the notebook. These criticisms reinforce prior work which has emphasized the negative impact of the lack of best practices of Software Engineering in scientific computing software [9], regarding separation of concerns [10], tests [11], and maintenance [12].

Existing work attempted to understand how notebooks are used [3], [13], [14]. They analyzed different aspects of notebooks, including use cases [13], narrative [3], [13], and structure [3], [14]. However, they did not attempt to run the notebooks and check characteristics related to reproducibility.

In this paper, we present a study that aims to provide insights into the reproducibility aspects of real notebooks. To better understand the different characteristics that impact reproducibility, using the aforementioned criticisms as a guide.

AI Adoption in 2024: **74%** of Companies **Struggle to Achieve and Scale Value**
BCG, Press Release, 2024

We estimate that as much as **90 percent** of the failures in ML development come not from developing poor models but from **poor productization practices** and the challenges of integrating the model with production data and business applications, which keep the model from scaling and performing as intended.
Rewired, McKinsey & Company

O que abordaremos?

Agenda

1. Objetivos de aprendizado:

- Após esse módulo o aluno será capaz de desenvolver um fluxo end-to-end de ciência de dados com código pronto para ser colocado em produção via Docker.

2. Requisitos:

- Conceitos de programação
- Python
- Machine Learning

3. Material de aula:

https://drive.google.com/drive/folders/1-d8HhcRKG8q7A0uAmUGGj2UAheWOFCCo?usp=drive_link

1. Introdução

- O que é deploy?

2. Production Ready Data Science with Kedro

- O que é código produtivo?
- Como desenvolver código produtivo com Kedro?

3. Deploying Pipelines with FastAPI & Docker

- Como preparar pipelines para servir como API?
- Como servir pipelines com Docker?

4. [Extra] Deploying Pipelines with GCP Cloud Services

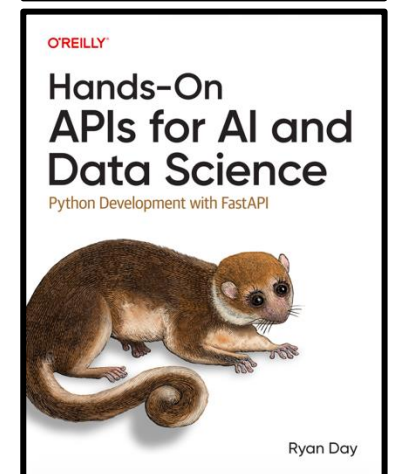
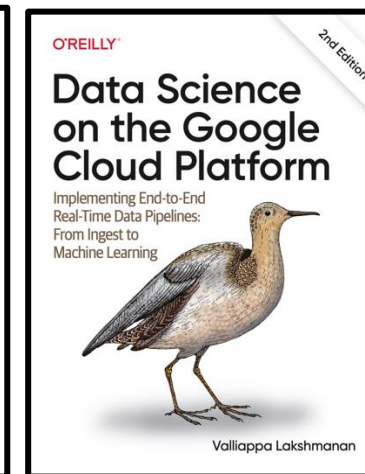
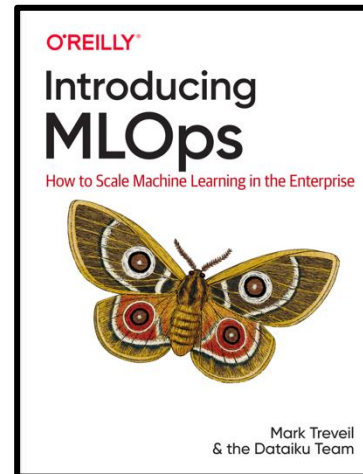
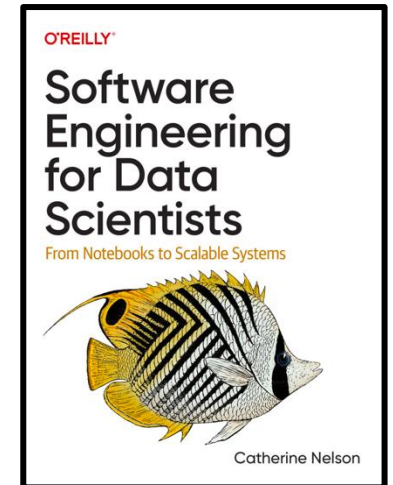
- Como servir pipelines com Docker na nuvem?

Onde aprofundar?

Material de Suporte

- Kedro:
 - <https://docs.kedro.org/en/stable/>
 - <https://www.youtube.com/@kedro-python>
- Docker:
 - <https://www.docker.com/>
 - https://www.youtube.com/playlist?list=PLkA60AVN3hh_nsDu5HtEqZ-xfjF-0-PfX
- FastAPI
 - <https://fastapi.tiangolo.com/>
- Google Cloud Platform
 - <https://cloud.google.com/>
- CRISP-DM:
 - <https://www.datascience-pm.com/crisp-dm-2/>

- Cheatsheets:
 - Na pasta: ./resources
 - cheatsheet-kedro.md
 - cheatsheet-kedro-fastapi.md
 - cheatsheet-docker.md
 - cheatsheet-kedro-api-gcp.md



O que preciso para começar?

Preparando o Ambiente – preparar antes da Aula

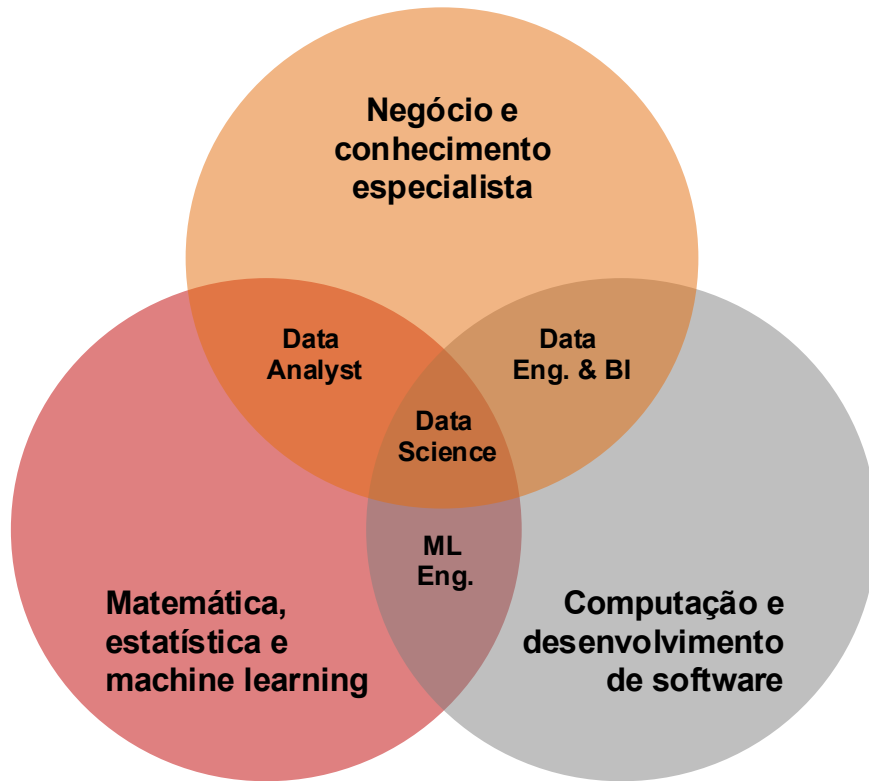
- Pacotes & Software necessários:
 - Conda/miniconda
 - Python v. 3.13.x
 - Kedro v. 1.2.x
 - Docker Desktop 4.6.x
 - FastAPI 0.128.x
- No Windows:
 - Docker requer WSL2
- Recomendado:
 - VSCode configurado como IDE
 - uv
 - [Extra] Conta Google Cloud Free Tier
- Tutoriais:
 - [resources/tutorials](#)



Introdução ao Deployment

O que é “deploy” no contexto de Ciência de Dados?

As diferentes perspectivas de uma solução digital



- Perspectiva de negócio:
 - Produto
- Perspectiva de computação:
 - Software
- Perspectiva matemática/estatística:
 - Solução analítica (incl. modelo)

Deploy é muito mais do que um processo técnico de mudança do ambiente de um novo ativo

Deployment

Deployment (ou promoção):

- No seu core, “deployment” é o processo através do qual um ativo – uma linha de código, um modelo de machine learning, uma nova feature – é promovido do ambiente de desenvolvimento (onde foi criado) para o ambiente de produção (onde será utilizado);
- É a ponte entre criação e geração de valor e envolve dimensões de software, analítica (de ciência de dados) e organizacionais;

Deployment vs MLOPs:

- **MLOPs:** *at its core, MLOps is the standardization and streamlining of machine learning life cycle management*¹
- MLOPs cobre o ciclo contínuo de desenvolvimento, testes, promoção e monitoramento de ativos de ML

Dimensões do Deployment:

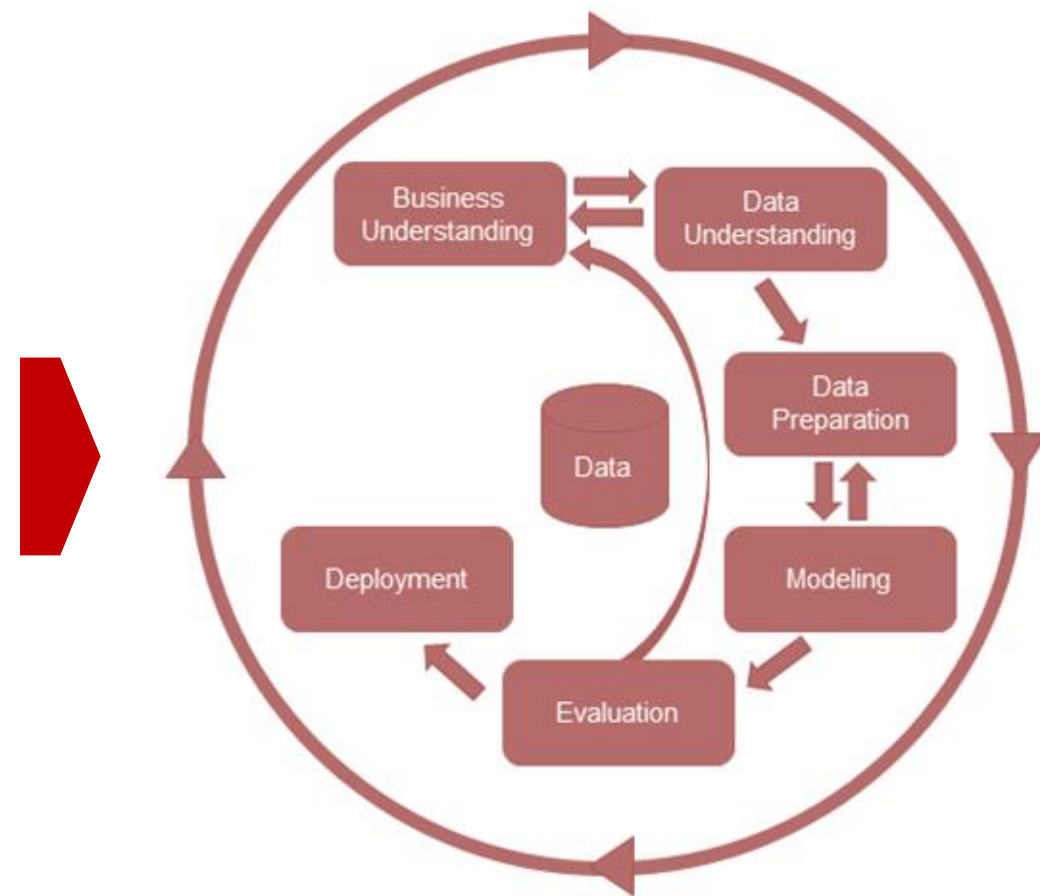
- Dimensão de **software**:
 - Como o novo ativo é tecnicamente promovido para produção e disponibilizada ao usuário?
- Dimensão **analítica**:
 - Como o novo ativo é estabilizado em produção?
- Dimensão **organizacional**:
 - É necessário ajustar o processo com o deploy do novo ativo?
 - Como o novo ativo será monitorado, mantido, evoluído?

1. Introducing MLOPs

Desta forma, a necessidade de promoção de um ativo de Machine Learning impõe ao ciclo de ciência de dados uma série de requisitos funcionais

Como deploy afeta o ciclo de ciência de dados?

- Requisitos para os componentes analíticos:
 - **Abordagem analítica:** define como a solução problema é decomposto em módulos interrelacionados
 - **Features do software:** como funcionalidades de treinamento, teste, backtest e inferência;
 - **Requisitos computacionais:** determinismo, reprodutibilidade, tempo de execução, etc.
 - **Requisitos de desempenho:** como precisão, recall, AUC, MAPE, etc.
 - **Requisitos de segurança:** como anonimização, localização, viés e segurança de modelos
 - **Requisitos de disponibilidade, escalabilidade e custos:** cobertura de testes, ferramentas, design patterns, linguagens e tech stack;
 - **Requisitos de manutenibilidade:** como equipe necessária, processos, recursos de monitoramento, etc.



Estes, por sua vez, exigem a utilização de boas práticas de engenharia de software no código do cientista

Como deploy afeta como o código é escrito?

- **Simplicidade:** escreva apenas uma vez, código reutilizável:
 - Escolha correta do paradigma de desenvolvimento (ex. funcional vs. orientação a objetos) e estrutura do projeto;
 - Datasets declarativos com abstração de I/O da lógica;
- **Modularidade:** decomponha o trabalho em pequenas funções com entradas e saídas claras e bem definidas:
 - Definição da abordagem analítica e dos módulos que a compõem;
- **Legibilidade:** facilite para o *você do futuro* (e para colegas) entender o código rapidamente:
 - Convenções, nomenclaturas, anotações, documentação, linting, code-formatters, etc.
- **Desempenho:** meça primeiro, otimize gargalos depois (não adivinhe):
 - Mensuração simples, caching, execução em paralelo, etc;
- **Robustez:** lide com entradas inesperadas, gere erros úteis e mantenha execuções reproduzíveis:
 - Logging, testes, unitários e integrados, como parte da definição de pronto do projeto;
 - Separação de parâmetros da lógica;
 - Facilidade de automação dos workflows de verificação do código;
- **Reprodutibilidade e gestão de dependências:**
 - Separação estrita de ambientes com configuração no ambiente e não no código;
 - Datasets versionados;

Como isso afeta seu trabalho?

Reflexão

- O que isso significa para você, como cientista de dados/líder de ciência de dados?
- O que impede que cientistas apliquem essas boas práticas em seu trabalho?
- Como trazer as boas práticas de software para o ciclo de ciência de dados?

...ou... substitua o engenheiro de software pelo Kedro!

Inspire



Deployment: Production-Ready Data Science

Prof. Donald Neumann, PhD.
Março de 2026

Contatos:
donaldn@insper.edu.br

Course overview: from notebook to production in 8 parts

- **Part 0: before class**
 - Prerequisites and environment setup (Python, uv, Ruff, Git)
- **Part 1:**
 - The problem: notebooks in production
- **Part 2:**
 - Conceptual framework: Why Kedro + FastAPI + Docker
- **Part 3:**
 - Step-by-step build guide (Data Eng., Modelling, Refit, Inference)
- **Part 4:**
 - Exposing pipelines as APIs with FastAPI
- **Part 5:**
 - Containerization with Docker
- **[Extra] Part 6:**
 - Deploying to Google Cloud Run
- **[Extra] Part 7:**
 - Development best practices (Ruff, pytest, logging)

Prerequisites and environment setup

Part 0: Setup

- **Python 3.13+**
 - `python --version` # verify 3.13.x
 - Use pyenv (Linux/macOS) or official installer (Windows)
- **uv - The modern Python package manager:**
 - `curl -LsSf https://astral.sh/uv/install.sh | sh` (or `pip install`)
 - Replaces `pip/pip-tools/poetry` - 10-100x faster
 - `uv.lock` pins every transitive dependency with hashes
- **IDE: Cursor or VS Code with Python extension**
 - Format on save (Ruff), lint on save (Ruff), interpreter -> `.venv`
- **Ruff - replaces flake8 + isort + black + pyupgrade:**
 - `uv run ruff check src/` # lint
 - `uv run ruff format src/` # format
- **Git: never commit data or credentials (.gitignore)**

Quick start: cloning and running the finished project

Part 0: Quick Start

- **See the big picture before building from scratch:**
- **Clone and install:**
 - `cd /path/to/project`
 - `uv sync`
- **Run the full Kedro pipeline:**
 - `uv run kedro run`
- **Start the FastAPI server:**
 - `uv run uvicorn churn.api:app --host 0.0.0.0 --port 8000`
 - Open Swagger docs at <http://localhost:8000/docs>
- **Or use Docker:**
 - `docker compose up --build`
- **This is our goal: one command to train, serve, and deploy**

The problem: why notebooks fail in production

Part 1: The Problem

- **Concrete problems with Kaggle-style notebooks:**
 - No reproducibility - random splits without seeds, cell order matters
 - No separation of concerns - cleaning, features, modelling interleaved
 - Hardcoded paths - `pd.read_csv('../data/telco.csv')` in code
 - No configuration management - hyperparameters scattered across cells
 - Data leakage - `fit()` on entire dataset before splitting
 - No API layer - no way for other systems to call the model
 - No deployment story - cannot run without Jupyter
 - No testing - manual 'looks right' validation
 - 'It works on my machine' - no lockfile, no containers

Data leakage: the silent killer in ML pipelines

Part 1: Data Leakage

- **What is data leakage?**
 - Fitting encoders/scalers on the ENTIRE dataset before splitting
 - The scaler learns mean/std of the test set - evaluation is biased
- **WRONG (notebook approach):**
 - `fit(all_data) -> split -> train/test`
- **RIGHT (production approach):**
 - `split -> fit(train_only) -> transform(all)`
- **Our solution with Kedro:**
 - Separate `fit_encoders()` and `transform_encoders()` functions
 - Configuration: `split_to_fit: ['train']` prevents leakage

Kedro: pipeline framework for production data science

Part 2: Kedro

▪ **What is Kedro?**

- Open-source Python framework by QuantumBlack (Linux Foundation)
- Define workflows as a DAG of nodes
- Data Catalog decouples I/O from logic
- Parameters in YAML, separated from code
- Pipeline visualization with kedro viz
- Node functions are pure Python - easy to test

▪ **Pros:**

- Opinionated structure - team follows same convention
- Built-in best practices (data layers, config separation)
- Pipeline viz for stakeholder communication

▪ **Cons:**

- Learning curve for abstractions
- Boilerplate for small projects
- Not an orchestrator (use Airflow/Prefect for scheduling)

FastAPI: modern Python web framework for ML APIs

Part 2: FastAPI

- **Key features for ML deployment:**
 - Automatic OpenAPI/Swagger docs
 - Pydantic validation for request/response
 - Async support for concurrent requests
 - Type safety catches errors early
 - Built on Starlette + Uvicorn (high performance)
- **Why not Flask?**
 - FastAPI gives auto docs + validation + type safety
 - Less code for the same functionality
 - Modern Python patterns (type hints, dataclasses)
- **Code snippet:**
 - `@app.post('/inference')`
 - `def run_inference(request: InferenceRequest):`
 - `# Pydantic validates input automatically`
 - `return InferenceResponse(predictions=...)`

Docker + uv: reproducible environments everywhere

Part 2: Docker + uv

- **Docker - Containerization:**

- Exact same environment runs everywhere
- No conflicts with host system Python
- Standard deployment unit across industry
- Composable with docker-compose
- Scalable via Kubernetes

- **uv - Modern Package Manager:**

- 10-100x faster than pip
- Deterministic lockfile (uv.lock) with hashes
- Built-in virtual environment management
- Compatible with pyproject.toml (PEP 621)

- **Quick start:**

- `curl -LsSf https://astral.sh/uv/install.sh | sh`
- `uv sync # install all deps from lockfile`

Kedro project structure: scaffolding a production project

Part 3: Project Scaffold

- **Scaffold with:** `uv run kedro new --name churn`
- **churn/**
 - `conf/base/` - Shared config (catalog.yml, parameters.yml)
 - `conf/local/` - Environment-specific (gitignored, credentials)
 - `data/01_raw/ ... 08_reporting/` - Data organized by processing stage
 - `src/churn/` - Source code package
 - `pipelines/` - Pipeline modules (data_engineering, modelling, ...)
 - `pipeline_registry.py` - Auto-discovers all pipelines
 - `settings.py` - Project settings (base_env, default_run_env)
 - `tests/` - Mirrors src/ structure
 - `pyproject.toml` - Dependencies + tool config
- **Key insight:**
 - Data layers (01_raw to 08_reporting) prevent accidental overwrites

Data engineering pipeline: the fit/transform pattern

Part 3: Data Engineering

- **Pipeline flow (DAG):**
 - raw_data -> [clean_data] -> [add_split_column] -> split_data
 - split_data -> [fit_encoders] -> encoders (saved as .pkl)
 - split_data + encoders -> [transform_encoders] -> encoded_data
 - encoded_data -> [fit_scalers] -> scalers (saved as .pkl)
 - encoded_data + scalers -> [transform_scalers] -> master_table
- **Core principle: separate fit and transform**
 - fit_encoders() learns from train split only
 - transform_encoders() applies to all splits
 - Artifacts (encoders, scalers) are saved for reuse in inference
- **Configuration-driven: split_to_fit: ['train']**

Kedro nodes: pure Python functions with no I/O

Part 3: Node Implementation

- **Node: `clean_data(raw_data, columns) -> DataFrame`**
 - `all_columns = [item for v in columns.values() ...]`
 - `df[c] = pd.to_numeric(df[c], errors='coerce').fillna(0)`
 - Key: never mutate input - use `.copy()`
- **Node: `fit_encoders(df, columns, params) -> dict[str, LabelEncoder]`**
 - `le.fit(df.loc[df['split'].isin(params['split_to_fit']), col])`
 - Returns dict mapping column names to fitted encoders
- **Node: `transform_encoders(df, encoders) -> DataFrame`**
 - `df[col] = encoder.transform(df[col].astype(str))`
- **Same pattern for `fit_scalers` / `transform_scalers` (StandardScaler)**

Wiring pipelines: nodes, catalog, and parameters

Part 3: Pipeline Wiring

- **Pipeline definition (pipeline.py):**
 - `Node(func=clean_data,`
 - `inputs=['raw_churn_data', 'params:columns'],`
 - `outputs='cleaned_churn_data', name='clean_data')`
- **Data Catalog (catalog.yml) - decouples I/O:**
 - `raw_churn_data:`
 - `type: pandas.CSVDataset`
 - `filepath: data/01_raw/telco-churn.csv`
 - `load_args: {sep: ';'}`
- **The magic:**
 - Node code never calls `pd.read_csv()` or `pd.to_csv()`
 - Swap CSV to Parquet, or local to S3, by changing YAML only
 - `'params:columns'` loads from `parameters.yml` automatically

Modelling pipeline: train, evaluate, and optimize

Part 3: Modelling

- **Dynamic class loading via YAML:**
 - `class_path: 'sklearn.linear_model.LogisticRegression'`
 - `cls = _load_class(params['class_path'])`
 - Switch models by changing YAML - no code changes
- **train_model() - generic training:**
 - `estimator = cls(**params.get('init_args', {}))`
 - `estimator.fit(X_train, y_train)`
 - Returns model artifact dict (estimator + metadata)
- **optimize_hyperparameters() - GridSearchCV:**
 - CatBoost with `param_grid: depth x iterations x learning_rate`
 - `cv=5, scoring='roc_auc', n_jobs=-1` (all CPU cores)
- **evaluate_model() reused for both baseline and optimized**

Refit and inference: from evaluation to production

Part 3: Refit and Inference

- **Why refit?**
 - During modelling: fit on train only (prevent leakage for evaluation)
 - For production: refit on ALL data (maximize model quality)
 - Safe because we already validated the approach
- **Refit pipeline reuses data_engineering nodes:**
 - `from churn.pipelines.data_engineering.nodes import fit_encoders`
 - Same functions, different params: `split_to_fit: ['train','test','validate']`
 - Outputs: `production_encoders`, `production_scalers`, `production_model`
- **Inference pipeline - maximum node reuse:**
 - Reuses: `clean_data`, `transform_encoders`, `transform_scalers`
 - Only 2 new functions: `to_dataframe()` and `predict()`
 - No fitting happens - only transform with production artifacts

FastAPI: wrapping Kedro pipelines as REST endpoints

Part 4: API Architecture

- **API endpoints:**
 - POST /train - Async: runs data_eng + modelling + refit
 - GET /train/{run_id} - Poll training status
 - POST /inference - Sync: online prediction from JSON
 - POST /batch-inference - Async: run inference from catalog
- **Bootstrapping Kedro in FastAPI:**
 - bootstrap_project(PROJECT_PATH)
 - configure_project('churn')
 - Thread-safe with double-checked locking pattern
- **Pydantic models as API contracts:**
 - class InferenceRequest(BaseModel):
 - instances: list[dict[str, Any]]
 - Auto-generated Swagger docs at /docs

Online inference: bridging HTTP and Kedro with MemoryDataset

Part 4: Online Inference

- **The key pattern - MemoryDataset override:**
 - with KedroSession.create(project_path=...) as session:
 - catalog = session.load_context().__get_catalog()
 - # Inject HTTP data into catalog
 - catalog['raw_inference_data'] = MemoryDataset(
 - data=pd.DataFrame(request.instances)))
 - catalog['inference_predictions'] = MemoryDataset()
 - # Run the SAME pipeline as batch mode
 - SequentialRunner().run(pipeline, catalog)
 - predictions = catalog.load('inference_predictions')
- **Same inference pipeline for batch (CSV) and online (HTTP)**

Docker: packaging the application for deployment

Part 5: Docker

- **Dockerfile - two-phase install for layer caching:**
 - FROM python:3.13-slim
 - COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/
 - COPY pyproject.toml uv.lock ./ # deps first (cached layer)
 - RUN uv sync --frozen --no-dev --no-install-project
 - COPY src/ src/ # source second
 - RUN uv sync --frozen --no-dev # install project
 - CMD ['uv','run','uvicorn','churn.api:app','--host','0.0.0.0']
- **docker-compose.yml:**
 - services:
 - api:
 - build: .
 - ports: ['8000:8000']
 - volumes:
 - ./conf:/app/conf:ro # config read-only
 - ./data:/app/data # data read-write

Deploying to Google Cloud Run: from local to production

Part 6: Cloud Run

- **Why Cloud Run?**
 - Fully managed, scale-to-zero, Docker-native, HTTPS by default
 - Generous free tier: 2M requests/month
- **Cloud catalog overlay (zero code changes!):**
 - `conf/cloud/catalog.yml` overrides filepaths to `gs://BUCKET/...`
 - `KEDRO_ENV=cloud` activates the cloud environment
- **Deploy commands:**
 - `gcloud builds submit --tag .../churn-api:latest`
 - `gcloud run deploy churn-api --image ... \`
 - `--set-env-vars KEDRO_ENV=cloud \`
 - `--allow-unauthenticated --memory 1Gi`
- **Key takeaway: local to cloud = only new catalog.yml**

Development best practices: Ruff, pytest, and logging

Part 7: Best Practices

- **Ruff - replaces flake8 + black + isort:**
 - `uv run ruff check src/` # lint
 - `uv run ruff format src/` # format
 - Rules: F (pyflakes), I (isort), UP (pyupgrade), PL (pylint), T201 (print)
- **pytest with coverage:**
 - `uv run pytest`
 - Test pyramid: unit tests for nodes, integration for pipelines, e2e for API
- **Structured logging:**
 - `logger = logging.getLogger(__name__)` # not print()!
 - Rich console output + rotating file handler (10MB, 20 backups)
- **Kedro environments:**
 - `conf/base/` (shared, committed) vs `conf/local/` (gitignored)
 - Credentials in `credentials.yml` - NEVER committed to git

Discussion and exercises

Part 8: Exercises

- **Discussion questions:**

- How can Kedro principles help your team?
- When is the fit/transform pattern critical?
- What are the tradeoffs of config-driven model selection?

- **Exercises:**

- 1. Add a /health endpoint to the FastAPI app
- 2. Add XGBoost as a third model (YAML + 2 pipeline nodes)
- 3. Write a unit test for clean_data with synthetic DataFrames
- 4. Add a new feature column and trace its path through all pipelines
- 5. (Advanced) Deploy to Cloud Run and run inference remotely

Inspire



Deployment: Production-Ready Data Science

Prof. Donald Neumann, PhD.
Março de 2026

Contatos:
donaldn@insper.edu.br

Previendo a incidência de Diabetes

Exercício

- **Considere:**

- O notebook diabetes-prediction.ipynb
- Os dados diabetes-dataset-*.csv em data

1. Planeje os pipelines kedro para esse notebook
2. Construa os pipelines de engenharia de dados, treinamento e inferência
3. [Extra] Exponha os pipelines como APIs utilizando FastAPI
4. [Extra] Containerize a o pipeline utilizando docker

Inspire